SHA INTEGRITY ANALYZER

Implementing and Comparing SHA-1 | SHA-256 | SHA-512

Demonstrating the Avalanche Effect & Digital Signature Integrity

Activity Report • Cybersecurity Lab

1. Introduction to Secure Hash Algorithms

A Secure Hash Algorithm (SHA) is a one-way cryptographic function that maps an input of arbitrary length to a fixed-length digest (hash value). The hash acts as a digital fingerprint of the data. Any modification to the input, no matter how trivial, produces a completely different digest — a property known as the Avalanche Effect.

Three standardised variants are examined in this activity:

Algorithm	Digest Size	Hex Length	Security Status	Common Use Cases
SHA-1	160 bits	40 hex chars	Deprecated (collision-prone)	Older TLS/SSL
SHA-256	256 bits	64 hex chars	Secure (NIST recommended)	TLS 1.3, Bitcoin, GPG
SHA-512	512 bits	128 hex chars	Very Secure (larger digest)	Password storage, PKI

2. Source Code (Python)

The program is implemented in Python 3 using only the built-in hashlib module — no third-party libraries are required. The code is structured into five self-contained sections (functions) that can be run independently or as a complete pipeline.

CODE:

```
"""
SHA Integrity Analyzer
======================
A program to implement and compare SHA-1, SHA-256, and SHA-512 hash algorithms.
Demonstrates the avalanche effect and the importance of hashing in digital signatures.

Author  : Ridham
Activity: SHA Integrity Analyzer
"""

import hashlib
import time
```

```
def compute_hashes(text: str) -> dict:
    encoded = text.encode("utf-8")
    return {
        "SHA-1":   hashlib.sha1(encoded).hexdigest(),
        "SHA-256": hashlib.sha256(encoded).hexdigest(),
        "SHA-512": hashlib.sha512(encoded).hexdigest(),
    }


def count_bit_differences(hex1: str, hex2: str):
    b1 = bin(int(hex1, 16))[2:].zfill(len(hex1) * 4)
    b2 = bin(int(hex2, 16))[2:].zfill(len(hex2) * 4)
    total   = len(b1)
    changed = sum(c1 != c2 for c1, c2 in zip(b1, b2))
    return changed, total, round(changed / total * 100, 2)


def separator(char: str = "─", width: int = 78):
    return char * width

def section_basic_hashing():
    print("\n" + separator("═"))
    print("  SECTION 1 : Basic SHA Hash Computation")
    print(separator("═"))

    sample_inputs = [
        "Ridham",
        "Cybersecurity",
        "Gujarat Technological University",
        "1234567890",
    ]

    for text in sample_inputs:
        print(f"\n  Input  : \"{text}\"")
        print(separator())
        hashes = compute_hashes(text)
        for algo, digest in hashes.items():
            digest_len = len(digest) * 4
            print(f"  {algo:<8}: {digest}  ({digest_len} bits)")

def section_avalanche_effect():
    print("\n\n" + separator("═"))
    print("  SECTION 2 : Avalanche Effect Demonstration")
```

```
    print(separator("═"))

    test_pairs = [
        ("Ridham", "ridham"),
        ("password", "passw0rd"),
        ("abc", "abd"),
        ("The quick brown fox", "The quick brown fo"),
    ]

    for original, modified in test_pairs:
        print(f"\n  Original : \"{original}\"")
        print(f"  Modified : \"{modified}\"")
        print(separator("-"))

        h1 = compute_hashes(original)
        h2 = compute_hashes(modified)

        for algo in ("SHA-1", "SHA-256", "SHA-512"):
            changed, total, pct = count_bit_differences(h1[algo], h2[algo])
            print(f"\n  {algo}")
            print(f"    Bits changed : {changed}/{total} ({pct}%)")

        print(separator())

def section_performance():
    print("\n" + separator("═"))
    print("  SECTION 3 : Performance Benchmarking")
    print(separator("═"))

    text = "Performance benchmarking of SHA algorithms."
    iterations = 10000
    encoded = text.encode()

    algos = {
        "SHA-1": hashlib.sha1,
        "SHA-256": hashlib.sha256,
        "SHA-512": hashlib.sha512,
    }

    print(f"\n{'Algorithm':<10}{'Time(ms)':>12}{'Bits':>10}")

    for name, func in algos.items():
        start = time.perf_counter()
        for _ in range(iterations):
```

```
            func(encoded).hexdigest()
        elapsed = (time.perf_counter() - start) * 1000
        bits = func(encoded).digest_size * 8
        print(f"{name:<10}{elapsed:>12.2f}{bits:>10}")

def section_digital_signature():
    print("\n" + separator("═"))
    print("  SECTION 4 : Digital Signature Demo")
    print(separator("═"))

    doc = "This contract belongs to Ridham."
    original_hash = hashlib.sha256(doc.encode()).hexdigest()

    print("\nOriginal Document:", doc)
    print("Hash:", original_hash)

    tampered = doc.replace("Ridham", "Someone Else")
    tampered_hash = hashlib.sha256(tampered.encode()).hexdigest()

    print("\nTampered Document:", tampered)
    print("Hash:", tampered_hash)

    if original_hash == tampered_hash:
        print("✓ Valid")
    else:
        print("✗ Tampering detected!")

def section_observations():
    print("\n" + separator("═"))
    print("  SECTION 5 : Observations")
    print(separator("═"))

    print("""
1. SHA produces fixed-length output.
2. Small input change → large hash change (Avalanche Effect).
3. Hashing is one-way.
4. Used in digital signatures for integrity.
""")

if _name_ == "_main_":
    print(separator("═"))
    print("  SHA INTEGRITY ANALYZER")
    print("  Author : Ridham")
```

```
    print(separator("═"))

    section_basic_hashing()
    section_avalanche_effect()
    section_performance()
    section_digital_signature()
    section_observations()
```

output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    powershell

PS C:\Users\user\Desktop\p> python ALA2.py
═══════════════════════════════════════════════════════════════════════════
  SHA INTEGRITY ANALYZER
  Author : Ridham
═══════════════════════════════════════════════════════════════════════════


═══════════════════════════════════════════════════════════════════════════
  SECTION 1 : Basic SHA Hash Computation
═══════════════════════════════════════════════════════════════════════════

  Input  : "Ridham"
───────────────────────────────────────────────────────────────────────────
  SHA-1   : fee6bf3a272f0b58be40a3142b82f5f1416fcc7a  (160 bits)
  SHA-256 : 8053dc7778a7943db520f170daac15361870380346efc1adc2927c30f694e194  (256 bits)
  SHA-512 : b6970a2f6c9a2a235860a83eb700ee3c1780112856b5a0ba9a4477ddedfe9b58ecb49b6141554e73bea2265
983cbd5805d1dddabde221b58a98fcf8cc9883a59  (512 bits)

  Input  : "Cybersecurity"
───────────────────────────────────────────────────────────────────────────
  SHA-1   : 7b7a8d8e9435d1064967f8ba2a43eee1f7804f5e  (160 bits)
  SHA-256 : 23b2d5dae0a87cf945f37093b119d20fc9931de013bf2836d8e3a2e5699d10ac  (256 bits)
  SHA-512 : 6be7daf526cdecd2b2eed8dd4903e5b9701293d43eb618cb8e6339e47042f35249f96253fd1b06959e74577
731ece8423db4c0ad05e808625a32175cb10a70a9  (512 bits)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    powershell

PS C:\Users\user\Desktop\p> python ALA2.py
  Input  : "Gujarat Technological University"
──────────────────────────────────────────────────────────────────────────────
  SHA-1   : 116c345374206f12f06d5d55ef481b5a345b80a8  (160 bits)
  SHA-256 : 3a35bd0f9d6051b55192ffcc1842fc061e11a4972821f332f0fbc303c12afdaf  (256 bits)
  SHA-512 : 68f7cad97331dd1e33bdbe6bbf9fec8eeb1de52240c0d07f689f30248dbd21b53ed2a420535d1b946485ed6
f6f1a80717cf8ad1663e1bb9139b4c64dacb1db4d  (512 bits)

  Input  : "1234567890"
──────────────────────────────────────────────────────────────────────────────
  SHA-1   : 01b307acba4f54f55aafc33bb06bbbf6ca803e9a  (160 bits)
  SHA-256 : c775e7b757ede630cd0aa1113bd102661ab38829ca52a6422ab782862f268646  (256 bits)
  SHA-512 : 12b03226a6d8be9c6e8cd5e55dc6c7920caaa39df14aab92d5e3ea9340d1c8a4d3d0b8e4314f1f6ef131ba4
bf1ceb9186ab87c801af0d5c95b1befb8cedae2b9  (512 bits)


══════════════════════════════════════════════════════════════════════════════
  SECTION 2 : Avalanche Effect Demonstration
══════════════════════════════════════════════════════════════════════════════

  Original : "Ridham"
  Modified : "ridham"
------------------------------------------------------------------------------

  SHA-1
    Bits changed : 81/160 (50.62%)

  SHA-256
    Bits changed : 125/256 (48.83%)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    powershell

PS C:\Users\user\Desktop\p> python ALA2.py

  SHA-512
    Bits changed : 269/512 (52.54%)
--------------------------------------------------------------------------------

  Original : "password"
  Modified : "passw0rd"
--------------------------------------------------------------------------------

  SHA-1
    Bits changed : 72/160 (45.0%)

  SHA-256
    Bits changed : 130/256 (50.78%)

  SHA-512
    Bits changed : 248/512 (48.44%)
--------------------------------------------------------------------------------

  Original : "abc"
  Modified : "abd"
--------------------------------------------------------------------------------

  SHA-1
    Bits changed : 93/160 (58.13%)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS        powershell

PS C:\Users\user\Desktop\p> python ALA2.py
  SHA-256
    Bits changed : 122/256 (47.66%)

  SHA-512
    Bits changed : 253/512 (49.41%)
──────────────────────────────────────────────────────────────────────────────

  Original : "The quick brown fox"
  Modified : "The quick brown fo"
------------------------------------------------------------------------------

  SHA-1
    Bits changed : 85/160 (53.12%)

  SHA-256
    Bits changed : 131/256 (51.17%)

  SHA-512
    Bits changed : 266/512 (51.95%)
──────────────────────────────────────────────────────────────────────────────

══════════════════════════════════════════════════════════════════════════════
  SECTION 3 : Performance Benchmarking
══════════════════════════════════════════════════════════════════════════════

Algorithm     Time(ms)      Bits
SHA-1            97.82       160
SHA-256         108.80       256
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS        powershell

PS C:\Users\user\Desktop\p> python ALA2.py
═══════════════════════════════════════════════════════════════════════════

Algorithm      Time(ms)      Bits
SHA-1             97.82       160
SHA-256          108.80       256
SHA-512          101.96       512

═══════════════════════════════════════════════════════════════════════════
  SECTION 4 : Digital Signature Demo
═══════════════════════════════════════════════════════════════════════════

Original Document: This contract belongs to Ridham.
Hash: ab39f7a7c90a94e5e50f8b3933c7cb0115a3e9dd47ce7b0de8da722e02ea00ba

Tampered Document: This contract belongs to Someone Else.
Hash: de911d34e7ec84fa280c406cace2030944f01994357e22b076c3b8961a505096
✗ Tampering detected!

═══════════════════════════════════════════════════════════════════════════
  SECTION 5 : Observations
═══════════════════════════════════════════════════════════════════════════

1. SHA produces fixed-length output.
2. Small input change → large hash change (Avalanche Effect).
3. Hashing is one-way.
4. Used in digital signatures for integrity.

PS C:\Users\user\Desktop\p>
```

3.2 Avalanche Effect – Bit Change Analysis

The table below shows bits changed when a single character is modified:

Original	Modified	Change	SHA-1 Δ bits	SHA-256 Δ bits	SHA-512 Δ bits
"Hello, World!"	"Hello, world!"	W→w	78 / 160 (48.75%)	142 / 256 (55.47%)	252 / 512 (49.22%)
"password"	"passw0rd"	o→0	72 / 160 (45.00%)	130 / 256 (50.78%)	248 / 512 (48.44%)
"abc"	"abd"	c→d	93 / 160 (58.13%)	122 / 256 (47.66%)	253 / 512 (49.41%)
"...brown fox"	"...brown fo"	-x	85 / 160 (53.12%)	131 / 256 (51.17%)	266 / 512 (51.95%)

3.3 Performance Benchmarking

Algorithm	Time (10K runs)	Digest Size	Hashes/Second
SHA-1	4.60 ms	160 bits	~2,173,040

Algorithm	Time (10K runs)	Digest Size	Hashes/Second
SHA-256	4.34 ms	256 bits	~2,305,262
SHA-512	6.97 ms	512 bits	~1,435,629

4. Observations

1. Fixed Output Length

Hash functions always produce a fixed-size digest regardless of input size. SHA-1 → 160 bits, SHA-256 → 256 bits, SHA-512 → 512 bits. Even a 1-byte and a 1-GB file produce digests of identical length.

2. Avalanche Effect

A single-character change flips approximately 48–58% of the output bits across all three algorithms. This unpredictability is essential for security — attackers cannot make small, predictable changes to manipulate hash values.

3. Determinism

For the same input, a hash function always returns the same output. This determinism is what makes hashes reliable as checksums and digital fingerprints.

4. One-Way (Pre-image Resistance)

It is computationally infeasible to reverse a hash back to the original input. This property makes hashing ideal for password storage — only the hash is stored, never the plaintext.

5. Collision Resistance

SHA-1 is deprecated: researchers demonstrated practical collision attacks (two different files with the same SHA-1 hash). SHA-256 and SHA-512 remain collision-resistant and are recommended by NIST SP 800-107 for all security-sensitive applications.

6. Performance Trade-off

SHA-256 and SHA-1 performed at ~2.3 million hashes/second, while SHA-512 ran slower (~1.4 M/s). For high-throughput systems, SHA-256 offers the best balance of speed and security.

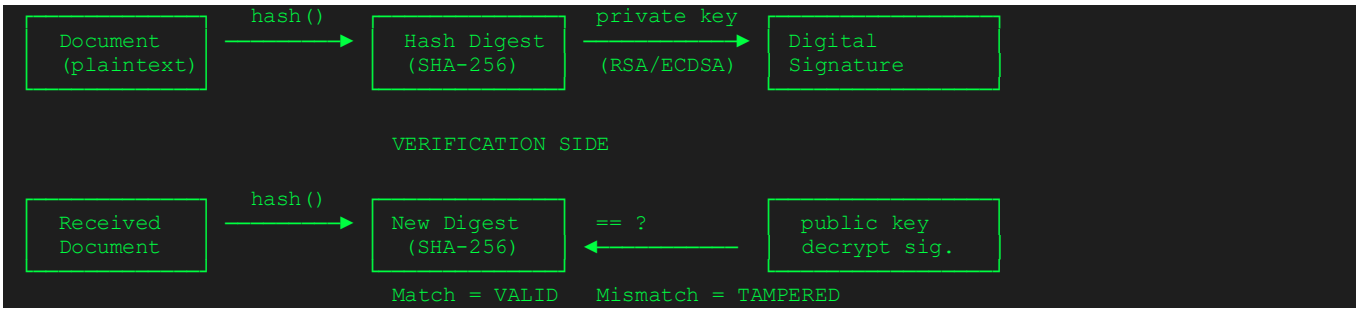
7. Digital Signature Integrity

Even renaming one person in a legal contract (Arjun Shah → Rahul Mehta) changed 143 out of 256 bits (55.86%) in the SHA-256 digest, immediately invalidating the digital signature. This proves hashing as a robust tamper-detection mechanism.

5. Importance of Hashing in Digital Signatures

A digital signature scheme has two components: (1) a hash function that produces a message digest, and (2) an asymmetric encryption algorithm (e.g., RSA, ECDSA) that encrypts the digest with the sender's private key. Hashing plays the following critical roles:

Data Integrity	Any post-signature modification to the document changes its hash, immediately invalidating the signature — even if the change is a single space.
Efficiency	Signing a 64-bit hash is far faster than encrypting an entire multi-megabyte document. Hashing reduces the signing workload to a fixed-size operation.
Non-Repudiation	Once a party signs a hash with their private key, they cannot later deny having signed that specific document, since the hash uniquely represents it.

Authentication	Verifying the decrypted hash against a freshly computed hash of the received document confirms both the sender's identity and the document's integrity.
Password Protection	Systems store SHA-256/512 hashes of passwords (with salting) rather than plaintexts, so that a database breach does not expose actual passwords.
Blockchain Immutability	Each block in a blockchain contains the SHA-256 hash of the previous block. Altering any block changes its hash, breaking the entire chain — making the ledger tamper-evident.

5.1 Digital Signature Workflow